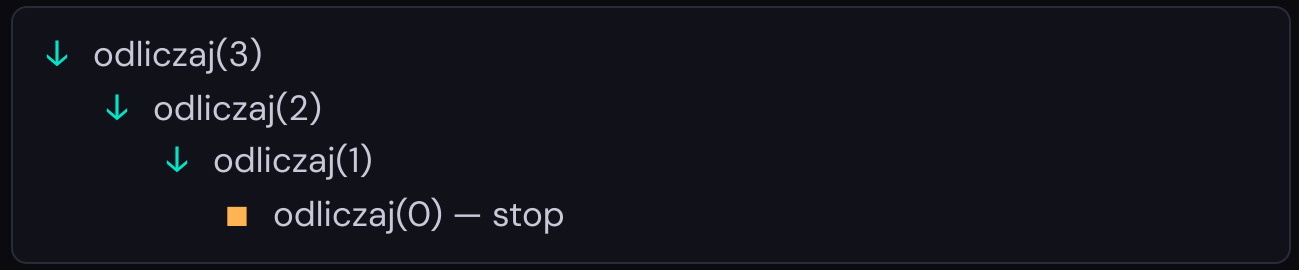
// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Rekurencja — pierwsze kroki

Funkcja, która wywołuje samą siebie — od zera, powoli i po kolei: warunek stopu, osobne kopie funkcji, stos wywołań i kod wykonywany w drodze w dół i w górę. Każde wywołanie śledzimy na osobnym slajdzie.

C++ · REKURENCJA · CZĘŚĆ 1 Z 6 · OD ZERA



PLAN PREZENTACJI

01 Przypomnienie: funkcje
02 Funkcja woła funkcję
03 Funkcja woła samą siebie
04 Warunek stopu
05 Śledzimy krok po kroku
06 Stos wywołań
07 Kod przed i po wywołaniu
08 Jak pisać rekurencję?
09 Więcej przykładów
10 Typowe błędy
11 Sprawdź się
12 Podsumowanie

01 Funkcja — krótkie przypomnienie

Funkcja to **kawałek programu z nazwą**. Piszemy go raz, a potem możemy go uruchamiać — **wywoływać** — ile razy chcemy.

DEFINICJA I DWA WYWOŁANIA

```
void powitaj() {
    cout << "Czesc!" << endl;
}

int main() {
    powitaj();
    powitaj();
}
```

NA EKRANIE

```
Czesc!
Czesc!
```

1. Linijki 1–3 to **definicja** — przepis. Sama definicja niczego nie robi.
2. Linijki 6 i 7 to **wywołania** — dopiero one uruchamiają przepis.
3. Dwa wywołania → napis pojawia się **dwa razy**.

01 Funkcja z parametrem

Funkcja może dostać **parametr** — wartość, którą podajemy przy wywołaniu w nawiasie.

```
void powitaj(string imie) {
    cout << "Czesc, " << imie << "!" << endl;
}

int main() {
    powitaj("Ola");
    powitaj("Bartek");
}
```

NA EKRANIE

```
Czesc, Ola!
Czesc, Bartek!
```

Wywołanie	Wartość imie
`powitaj("Ola")`	**"Ola"**
`powitaj("Bartek")`	**"Bartek"**

Każde wywołanie dostaje **własną** zmienną **imie** z własną wartością. Za chwilę to będzie bardzo ważne — zapamiętaj to zdanie.

01 Co dokładnie dzieje się przy wywołaniu?

1. **Program dochodzi do linijki z wywołaniem i zatrzymuje się w tym miejscu — zapamiętuje, gdzie skończył.**
2. **Przechodzi do funkcji. Parametr dostaje wartość podaną w nawiasie.**
3. **Wykonuje kolejne linijki funkcji — aż do jej końca albo do return.**
4. **Wraca dokładnie w to miejsce, w którym się zatrzymał, i idzie dalej od następnej linijki.**

Te cztery kroki działają zawsze — dla każdej funkcji, także dla funkcji rekurencyjnych. Cała rekurencja to tylko **wiele razy powtórzone te same cztery kroki**.

01 Śledzimy: w jakiej kolejności wykonują się linijki?

```
void f() {
    cout << "B";
}

int main() {
    cout << "A";
    f();
    cout << "C";
}
```

NA EKRANIE

```
ABC
```

Krok	Linijka	Co się dzieje
1	6	wypisujemy **A**
2	7	wywołanie: **main zatrzymuje się** i skacze do f
3	2	wypisujemy **B**
4	3	koniec f → **powrót** do main, za linijkę 7
5	8	wypisujemy **C**

01 Funkcja, która zwraca wynik

```
int kwadrat(int x) {
    return x * x;
}

int main() {
    int a = kwadrat(5);
    cout << a + kwadrat(3);
}
```

NA EKRANIE

```
34
```

Funkcja typu **int** oddaje wynik przez **return**. Wywołanie **zamienia się** w zwróconą liczbę:

1. `kwadrat(5)` zwraca **25**, więc `a = 25`.
2. `kwadrat(3)` zwraca **9**.
3. Na ekran trafia `25 + 9` = **34**.

> W tej części będziemy pisać głównie funkcje **void**, które coś wypisują. Funkcjami zwracającymi wynik zajmie się część 2.

02 Funkcja może wywołać inną funkcję

```
void c() {
    cout << "C ";
}
void b() {
    cout << "B ";
    c();
    cout << "b ";
}
void a() {
    cout << "A ";
    b();
    cout << "a ";
}
int main() {
    a();
}
```

Funkcja **a** woła **b**, a **b** woła **c**. Co pojawi się na ekranie?

Zastosuj cztery kroki z poprzedniej części: przy każdym wywołaniu funkcja **zatrzymuje się** i czeka, aż wywołana funkcja skończy.

NA EKRANIE

A B C b a

Małe litery **b** i **a** pojawiają się na końcu — bo b i a mogą dokończyć swoją pracę dopiero **po powrocie** z funkcji, którą wywołały.

02 Śledzimy: kto na kogo czeka?

PRZEBIEG PROGRAMU

↓ main woła `a()`
- ▸ a wypisuje **A**
- ↓ a woła `b()` — a czeka
  - ▸ b wypisuje **B**
  - ↓ b woła `c()` — b czeka
    - ▸ c wypisuje **C**
    - ↑ c kończy → wracamy do b
  - ▸ b dokańcza: wypisuje **b**
  - ↑ b kończy → wracamy do a
- ▸ a dokańcza: wypisuje **a**
- ↑ a kończy → wracamy do main

Wcięcie pokazuje, **jak głęboko** jesteśmy: im dalej w prawo, tym więcej funkcji czeka na swoją kolej.

1. ↓ — wywołanie, schodzimy głębiej.
2. ▸ — funkcja coś wypisuje.
3. ↑ — funkcja się kończy, wracamy wyżej.

Funkcje kończą się w **odwrotnej kolejności**, niż się zaczynały: zaczęły a, b, c — skończyły c, b, a.

02 Zasada powrotu

Gdy funkcja się kończy, wracamy tam, skąd ją wywołano — i stamtąd idziemy dalej.

Funkcja nie zapomina

Funkcja, która kogoś wywołała, **czeka**. Pamięta, w której linijce skończyła, i jakie miała wartości zmiennych.

Ostatnia zaczęta — pierwsza skończona

Najpierw kończy się funkcja wywołana **najpóźniej**. Dopiero potem ta, która na nią czekała.

02 Stos wywołań — pierwsze spojrzenie



Komputer zapisuje czekające funkcje jedna na drugiej — jak **stos talerzy**. To jest **stos wywołań**.

1. Wywołanie funkcji = **kładziemy** nowy talerz na wierzch.
2. Koniec funkcji = **zdejmujemy** talerz z wierzchu.
3. Działa zawsze tylko funkcja **z wierzchu**. Reszta czeka.

Obrazek pokazuje chwilę, gdy działa **c()** i wypisuje literę C.

03 A gdyby funkcja wywołała samą siebie?

```
void f() {
    cout << "Hej ";
    f();
}

int main() {
    f();
}
```

W linijce 3 funkcja **f** wywołuje… funkcję **f**. W C++ wolno tak napisać. Co się stanie?

NA EKRANIE

```
Hej Hej Hej Hej Hej Hej Hej Hej Hej Hej
Hej Hej Hej Hej Hej Hej Hej Hej Hej Hej
Hej Hej Hej Hej …
Segmentation fault
```

Uwaga: Każde f() woła kolejne f(), które woła kolejne f()… Nic tego nie zatrzymuje. Po chwili brakuje pamięci na kolejne czekające funkcje i **program się wysypuje**.

03 Rekurencja

Rekurencja to sytuacja, w której funkcja wywołuje samą siebie.

Brzmi jak błąd z poprzedniego slajdu — i bez zabezpieczenia rzeczywiście nim jest. Ale dobrze napisana rekurencja to jedno z najpotężniejszych narzędzi w programowaniu olimpijskim. Na czym polega „dobrze napisana”? Za chwilę.

Matrioszka

W każdej lalce jest mniejsza lalka. Ale w najmniejszej **nie ma już nic** — tam się kończy.

Lustro w lustrze

Dwa lustra naprzeciw siebie pokazują odbicie odbicia odbicia… To rekurencja **bez końca**.

Kolejka w sklepiku

Pytasz osobę przed sobą: „które jesteś?”. Ona pyta osobę przed nią… Zaraz to rozpiszemy.

03 Analogia: kolejka do sklepiku

PYTANIE IDZIE W DÓŁ, ODPOWIEDŹ WRACA W GÓRĘ

↓ Ty pytasz Kasię: „które jesteś w kolejce?”
  ↓ Kasia nie wie — pyta Olka
    ↓ Olek nie wie — pyta Zosię
      ■ Zosia stoi pierwsza: **„jestem 1.”**
    ↑ Olek: „to ja jestem **2.**”
  ↑ Kasia: „to ja jestem **3.**”
↑ Ty: „czyli ja jestem **4.**”

Każda osoba robi **to samo**: jeśli nie zna odpowiedzi, zadaje **to samo pytanie** osobie przed sobą, czeka na odpowiedź i dodaje do niej 1.

Tylko pierwsza osoba odpowiada **od razu**, bez pytania — to zatrzymuje całą lawinę pytań.

> To jest dokładnie rekurencja: **to samo pytanie, tylko mniejsze**, aż do przypadku, który znamy od ręki.

03 Każde wywołanie to osobna kopia funkcji

```
void f(int n) {
    cout << "zaczynam f(" << n << ")" << endl;
    if (n > 1) f(n - 1);
    cout << "koncze f(" << n << ")" << endl;
}
```

Wywołanie `f(3)` woła `f(2)`. Czy `f(2)` „psuje” zmienną **n** w `f(3)`? **Nie!**

kopia 1: f(3)
własne **n = 3**

kopia 2: f(2)
własne **n = 2**

kopia 3: f(1)
własne **n = 1**

NA EKRANIE

```
zaczynam f(3)
zaczynam f(2)
zaczynam f(1)
koncze f(1)
koncze f(2)
koncze f(3)
```

03 Kopie funkcji — najważniejsze

Każde wywołanie ma **własne** parametry i zmienne. Wywołania **nie widzą** nawzajem swoich zmiennych.

Pamiętasz zdanie z początku? „Każde wywołanie dostaje własną zmienną.” Przy rekurencji jest tak samo: gdy **f(3)** wywołuje **f(2)**, to powstaje nowa kopia funkcji z nowym **n**. Gdy ta kopia się skończy, wracamy do **f(3)** — a tam **n** nadal wynosi **3**.

> Dlatego po wypisaniu „koncze f(1)” program wypisał „koncze f(2)”, a nie znowu „koncze f(1)”: każda kopia pamięta **swoje** n.

04 Problem: rekurencja bez końca

```
void odliczaj(int n) {
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
}
```

Chcemy odliczać: **3 2 1**. Funkcja wypisuje n i woła siebie dla n − 1. Brzmi dobrze…

NA EKRANIE

```
3 2 1 0 -1 -2 -3 -4 -5 -6 -7 -8 …
Segmentation fault
```

Uwaga: …ale nic nie mówi funkcji, **kiedy przestać**. Po 1 przychodzi 0, potem −1, −2, i tak bez końca — aż program się wysypie.

04 Warunek stopu

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}
```

Dodajemy jedną linijkę: `if (n == 0) return;`

Gdy n dojdzie do zera, funkcja **od razu kończy pracę** — nie wypisuje nic i nie woła siebie dalej. To jest **warunek stopu** (po angielsku: *base case*).

NA EKRANIE

```
3 2 1
```

Teraz wywołania idą 3 → 2 → 1 → 0, a **odliczaj(0)** zatrzymuje lawinę.

04 Budowa funkcji rekurencyjnej

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}
```

1 · warunek stopu

`if (n == 0) return;` — najmniejszy przypadek, w którym **nie trzeba** wołać siebie. Piszemy go **na samej górze**.

2 · praca tego wywołania

`cout << n << " ";` — to, co robi **ta jedna kopia** funkcji: wypisuje swoje n.

3 · mniejszy problem

`odliczaj(n - 1);` — resztę pracy zlecamy kolejnej kopii, z **mniejszym** n. Każde wywołanie zbliża nas do warunku stopu.

04 Odliczanie — cały program

```cpp
#include <bits/stdc++.h>
using namespace std;

void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
    cout << "START" << endl;
}
```

NA EKRANIE

```
3 2 1 START
```

Napis START wypisuje **main** — dopiero wtedy, gdy `odliczaj(3)` całkowicie się skończy, razem ze wszystkimi kopiami, które wywołało.

> Na kolejnych slajdach prześledzimy ten program **wywołanie po wywołaniu** — razem ze stosem.

05 Start: main woła `odliczaj(3)`

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
    cout << "START";
}
```

NA EKRANIE

linijka	8
robimy	`odliczaj(3)` — main **zatrzymuje się** i czeka

STOS WYWOŁAŃ

`odliczaj(3)` zaczyna, n = 3 ← wierzch

`main()` czeka w linijce 8

DNO STOSU

05 Kopia `odliczaj(3)`: n = 3

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
    cout << "START";
}
```

warunek	`3 == 0` → **NIE** — idziemy dalej
wypisuje	**3**
woła	`odliczaj(2)` i czeka w linijce 4

STOS WYWOŁAŃ

- **`odliczaj(2)`** zaczyna, n = 2 ← wierzch
- **`odliczaj(3)`** n = 3, czeka w linijce 4
- **`main()`** czeka w linijce 8

DNO STOSU

NA EKRANIE

```
3
```

05 Kopia odliczaj(2): n = 2

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
    cout << "START";
}
```

NA EKRANIE

```
3 2
```

warunek	`2 == 0` → **NIE**
wypisuje	**2**
woła	`odliczaj(1)` i czeka

STOS WYWOŁAŃ

- **odliczaj(1)** zaczyna, n = 1 ← wierzch
- **odliczaj(2)** n = 2, czeka w linijce 4
- **odliczaj(3)** n = 3, czeka
- **main()** czeka w linijce 8

DNO STOSU

05 Kopia `odliczaj(1)`: n = 1

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
    cout << "START";
}
```

NA EKRANIE

```
3 2 1
```

warunek	`1 == 0` → **NIE**
wypisuje	**1**
woła	`odliczaj(0)` i czeka

STOS WYWOŁAŃ

- **odliczaj(0)** zaczyna, n = 0 ← wierzch
- **odliczaj(1)** n = 1, czeka w linijce 4
- **odliczaj(2)** n = 2, czeka
- **odliczaj(3)** n = 3, czeka
- **main()** czeka

DNO STOSU

05 Kopia `odliczaj(0)`: warunek stopu!

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
    cout << "START";
}
```

NA EKRANIE

```
3 2 1
```

warunek	`0 == 0` → **TAK**
robimy	`return` — koniec tej kopii
nie robi	nic nie wypisuje, **nikogo nie woła**

STOS WYWOŁAŃ

- **odliczaj(0)** return — znika ze stosu ← wierzch
- **odliczaj(1)** n = 1, czeka
- **odliczaj(2)** n = 2, czeka
- **odliczaj(3)** n = 3, czeka
- **main()** czeka

DNO STOSU

05 Powrót do odliczaj(1)

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
    cout << "START";
}
```

NA EKRANIE

```
3 2 1
```

gdzie	w linijce 4, zaraz po wywołaniu
co dalej	za wywołaniem nie ma już żadnych poleceń
robimy	dochodzimy do końca funkcji → **powrót**

STOS WYWOŁAŃ

- **odliczaj(1)** n = 1, kończy pracę ← wierzch
- **odliczaj(2)** n = 2, czeka
- **odliczaj(3)** n = 3, czeka
- **main()** czeka

DNO STOSU

05 Powrót do `odliczaj(2)` i `odliczaj(3)`

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
    cout << "START";
}
```

NA EKRANIE

```
3 2 1
```

`odliczaj(2)`	też nie ma nic po wywołaniu → kończy
`odliczaj(3)`	tak samo → kończy
stos	zdejmujemy kopie **od wierzchu**: 1, potem 2, potem 3

STOS WYWOŁAŃ

`odliczaj(3)` kończy pracę ← wierzch

`main()` czeka

DNO STOSU

05 Powrót do main: START

```
void odliczaj(int n) {
    if (n == 0) return;
    cout << n << " ";
    odliczaj(n - 1);
}

int main() {
    odliczaj(3);
    cout << "START";
}
```

NA EKRANIE

```
3 2 1 START
```

gdzie	main wraca za linijkę 8
robimy	`cout << "START"`
stos	został sam main — wszystkie kopie odliczaj zniknęły

STOS WYWOŁAŃ

main() działa dalej, linijka 9 ← wierzch

DNO STOSU

05 Całe odliczanie w jednej tabeli

Kopia	n	n == 0?	Wypisuje	Co dalej
`odliczaj(3)`	3	NIE	**3**	woła `odliczaj(2)`
`odliczaj(2)`	2	NIE	**2**	woła `odliczaj(1)`
`odliczaj(1)`	1	NIE	**1**	woła `odliczaj(0)`
`odliczaj(0)`	0	TAK	—	**return** — warunek stopu

W drodze **w dół** (3 → 2 → 1 → 0) każda kopia wypisuje swoje n. W drodze **w górę** (0 → 1 → 2 → 3) kopie tylko się kończą, bo po wywołaniu nie mają już nic do zrobienia. Za chwilę to zmienimy!

06 Stos — jak stos talerzy

`wywołanie funkcji`

kładziemy nowy talerz **na wierzch** stosu

`return / koniec funkcji`

zdejmujemy talerz **z wierzchu** stosu

`kto działa?`

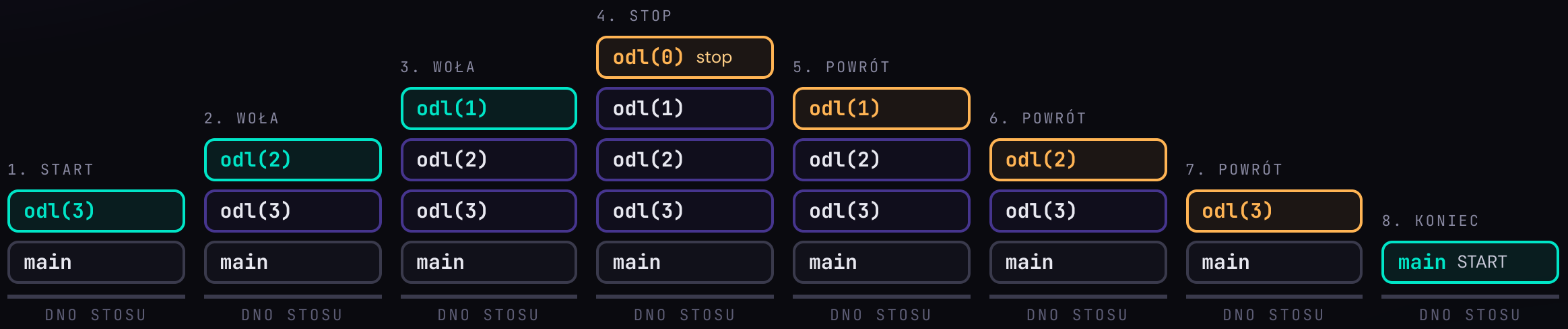
zawsze funkcja z **wierzchu** — pozostałe czekają

`na dnie`

zawsze leży **main** — program kończy się, gdy zdejmiemy i jego



06 Stos podczas `odliczaj(3)` — klatka po klatce



Stos najpierw **rośnie** (każde wywołanie dokłada kopię), a od warunku stopu **maleje** (każdy powrót zdejmuje kopię). Najwyżej był na 5 piętrach: main + 4 kopie odliczaj.

06 Ile miejsca zajmuje stos?

Każda czekająca kopia funkcji zajmuje trochę pamięci: swoje parametry, zmienne i informację, dokąd wrócić.

Wywołanie	Pięter stosu	Czy się zmieści?
`odliczaj(10)`	11	**bez problemu**
`odliczaj(10000)`	10 001	**tak**
`odliczaj(1000000)`	milion	zależy od komputera — **ryzykownie**
rekurencja bez stopu	nieskończenie wiele	**nigdy**

Uwaga: Gdy pięter jest za dużo, kończy się pamięć przeznaczona na stos. To jest **przepełnienie stosu** (ang. *stack overflow*) — program kończy się błędem.

Bezpieczna zasada na olimpiadzie: rekurencja o głębokości do **ok. 100 000** zwykle działa. Jeśli ma być głębsza — lepiej użyć pętli.

07 Przenieśmy jedną linijkę

WERSJA A — WYPISZ, POTEM WYWOŁAJ

```
void A(int n) {
    if (n == 0) return;
    cout << n << " ";
    A(n - 1);
}
```

A(3) WYPISUJE

```
3 2 1
```

WERSJA B — WYWOŁAJ, POTEM WYPISZ

```
void B(int n) {
    if (n == 0) return;
    B(n - 1);
    cout << n << " ";
}
```

B(3) WYPISUJE

```
1 2 3
```

Funkcje różnią się tylko **kolejnością dwóch linijek**, a wypisują liczby w odwrotnej kolejności. Dlaczego?

07 Wersja B: droga w dół

```
void B(int n) {
    if (n == 0) return;
    B(n - 1);
    cout << n << " ";
}
```

NA EKRANIE



↓ `B(3)` : 3 ≠ 0, woła `B(2)` — **jeszcze nic nie wypisuje**

↓ `B(2)` : woła `B(1)` — czeka

↓ `B(1)` : woła `B(0)` — czeka

■ `B(0)` : warunek stopu, **return**

W drodze w dół każda kopia zatrzymuje się na linijce 3 — na wywołaniu. Do `cout` w linijce 4 jeszcze **nikt nie doszedł**, więc ekran jest pusty.

07 Wersja B: droga w górę

```
void B(int n) {
    if (n == 0) return;
    B(n - 1);
    cout << n << " ";
}
```

NA EKRANIE

```
1 2 3
```

↑ `B(0)` wraca do `B(1)`
▸ `B(1)` dochodzi do linijki 4: wypisuje **1**
↑ `B(1)` wraca do `B(2)`
▸ `B(2)` wypisuje **2**
↑ `B(2)` wraca do `B(3)`
▸ `B(3)` wypisuje **3**

Kopie wracają w odwrotnej kolejności (1, 2, 3) — i dopiero **po powrocie** każda wypisuje swoje n.

07 Najważniejsza zasada rekurencji

Kod PRZED wywołaniem wykonuje się w drodze w dół. Kod PO wywołaniu — w drodze w górę, w odwrotnej kolejności.

W DÓŁ: 3, 2, 1

- ▸ przed: n = 3
  - ▸ przed: n = 2
    - ▸ przed: n = 1
      - ■ stop

W GÓRĘ: 1, 2, 3

- ■ stop
- ↑ po: n = 1
- ↑ po: n = 2
- ↑ po: n = 3

07 Przed i po — naraz

```
void f(int n) {
    if (n == 0) return;
    cout << n << " ";
    f(n - 1);
    cout << n << " ";
}
```

F(3) WYPISUJE

```
3 2 1 1 2 3
```

KOLEJNOŚĆ WYPISYWANIA

- ▸ f(3) przed: **3**
  - ▸ f(2) przed: **2**
    - ▸ f(1) przed: **1**
      - ■ f(0): stop
    - ↑ f(1) po: **1**
  - ↑ f(2) po: **2**
- ↑ f(3) po: **3**

07 Ćwiczenie: co wypisze f(3)?

```
void f(int n) {
    if (n == 0) return;
    cout << "(";
    f(n - 1);
    cout << ")";
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej. Podpowiedź: co wypisuje się w drodze w dół, a co w drodze w górę?

07 Odpowiedź: ((()))

- ▸ f(3) przed: (
  - ▸ f(2) przed: (
    - ▸ f(1) przed: (
      - ■ f(0): stop
    - ↑ f(1) po:)
  - ↑ f(2) po:)
- ↑ f(3) po:)

NA EKRANIE

```
((()))
```

Trzy nawiasy otwierające w drodze w dół, trzy zamykające w drodze w górę. Każde wywołanie „otacza” nawiasami wszystko, co wypisało wywołanie głębiej.

08 Trzy pytania przed napisaniem rekurencji

1. **Kiedy kończę? Jaki jest najmniejszy przypadek, który umiem obsłużyć od ręki, bez wołania siebie? To będzie warunek stopu.**

2. **Jak zmniejszyć problem? Z jakim argumentem wywołać siebie, żeby problem był mniejszy i zbliżał się do warunku stopu? (n − 1, n / 10, i + 1, …)**

3. **Co robi to jedno wywołanie — i kiedy? Jaka jest „moja część" pracy i czy wykonać ją przed wywołaniem (w dół), czy po nim (w górę)?**

> Te trzy pytania zadawaj sobie przy **każdym** zadaniu z rekurencji — także w kolejnych częściach kursu.

08 Przykład: wypisz liczby od 1 do n

Pytanie	Odpowiedź
Kiedy kończę?	gdy **n == 0** — nie ma nic do wypisania
Jak zmniejszyć?	wywołuję `wypisz(n - 1)`
Moja część?	wypisać **n**
Kiedy?	n ma być **na końcu**, więc **po** wywołaniu

```
void wypisz(int n) {
    if (n == 0) return;
    wypisz(n - 1);
    cout << n << " ";
}
```

WYPISZ(5) WYPISUJE

```
1 2 3 4 5
```

08 Zaufaj rekurencji

Nie śledź w głowie wszystkich wywołań. Załóż, że mniejsze wywołanie działa — i dopisz tylko swoją część.

Myślimy tak: „`wypisz(n - 1)` na pewno wypisze 1, 2, …, n − 1. Mnie zostaje dopisać **n**.” Koniec. Nie trzeba schodzić do zera.

`wypisz(5)` =
`wypisz(4)` **[czyli 1 2 3 4]** + wypisanie **5**

`wypisz(4)` =
`wypisz(3)` **[czyli 1 2 3]** + wypisanie **4**

`wypisz(0)` =
nic — warunek stopu

09 Przykład: napis n razy

```
void powtorz(int n, string s) {
    if (n == 0) return;
    cout << s << endl;
    powtorz(n - 1, s);
}

int main() {
    powtorz(3, "rekurencja");
}
```

NA EKRANIE

```
rekurencja
rekurencja
rekurencja
```

Pytanie	Odpowiedź
Kiedy kończę?	gdy n == 0
Jak zmniejszyć?	`powtorz(n - 1, s)`
Moja część?	wypisać s jeden raz

Funkcja może mieć **kilka parametrów**. Parametr s przekazujemy dalej bez zmian — zmniejsza się tylko n.

09 Przykład: liczby od a do b

```
void wypisz(int a, int b) {
    if (a > b) return;
    cout << a << " ";
    wypisz(a + 1, b);
}
```

WYPISZ(3, 6) WYPISUJE

```
3 4 5 6
```

- wypisz(3, 6): **3**
  - wypisz(4, 6): **4**
    - wypisz(5, 6): **5**
      - wypisz(6, 6): **6**
        - wypisz(7, 6): 7 > 6, stop

Tutaj problem **maleje**, choć liczba a **rośnie**: maleje odległość od a do b. Warunek stopu zależy od obu parametrów.

09 Przykład: co druga liczba

```
void coDruga(int n) {
    if (n <= 0) return;
    cout << n << " ";
    coDruga(n - 2);
}
```

CODRUGA(9) WYPISUJE

```
9 7 5 3 1
```

Uwaga: Z warunkiem `n == 0` ta funkcja by się wysypała! Dla n = 9 wywołania idą 9, 7, 5, 3, 1, **−1**, −3, … i **nigdy** nie trafiają w zero.

Warunek `n <= 0` łapie i zero, i liczby ujemne. Pisz warunek stopu tak, żeby **nie dało się go przeskoczyć**.

09 Przykład: linia gwiazdek

```
void linia(int k) {
    if (k == 0) return;
    cout << '*';
    linia(k - 1);
}
```

LINIA(5) WYPISUJE

```
*****
```

To ta sama funkcja co `powtorz`, tylko zamiast słowa wypisuje gwiazdkę i **bez** przechodzenia do nowej linii.

> Za chwilę użyjemy jej jak klocka: z wielu linii zbudujemy trójkąt. Rekurencyjne funkcje mogą wołać **inne** rekurencyjne funkcje.

09 Przykład: trójkąt z gwiazdek

```
void linia(int k) {
    if (k == 0) return;
    cout << '*';
    linia(k - 1);
}

void trojkat(int n) {
    if (n == 0) return;
    trojkat(n - 1);
    linia(n);
    cout << endl;
}
```

TROJKAT(4) WYPISUJE

```
*
**
***
****
```

Zaufaj rekurencji: `trojkat(n - 1)` narysuje trójkąt o wysokości n − 1. Nam zostaje dorysować **pod nim** wiersz z n gwiazdkami — czyli **po** wywołaniu.

09 Trójkąt krok po kroku

- ↓ trojkat(3) woła trojkat(2)
  - ↓ trojkat(2) woła trojkat(1)
    - ↓ trojkat(1) woła trojkat(0)
      - ■ trojkat(0): stop
    - ▸ trojkat(1) po powrocie: linia(1) → *
  - ▸ trojkat(2) po powrocie: linia(2) → **
- ▸ trojkat(3) po powrocie: linia(3) → ***

NA EKRANIE

```
*
**
***
```

A gdyby linię narysować **przed** wywołaniem? Wiersze pojawiłyby się w drodze w dół: ***, **, * — trójkąt do góry nogami. Sprawdź to w zadaniu „Odwrócony trójkąt”.

09 Przykład: cyfry liczby od końca

```
void cyfry(int n) {
    if (n < 10) {
        cout << n;
        return;
    }
    cout << n % 10 << " ";
    cyfry(n / 10);
}
```

Wywołanie	n % 10	n / 10
`cyfry(4821)`	wypisuje **1**	woła cyfry(482)
`cyfry(482)`	wypisuje **2**	woła cyfry(48)
`cyfry(48)`	wypisuje **8**	woła cyfry(4)
`cyfry(4)`	4 < 10 → wypisuje **4**	stop

CYFRY(4821) WYPISUJE

```
1 2 8 4
```

`n % 10` to ostatnia cyfra, a `n / 10` to liczba bez ostatniej cyfry. Problem maleje: liczba ma za każdym razem o jedną cyfrę mniej.

09 Przykład: cyfry od początku

```
void cyfry(int n) {
    if (n < 10) {
        cout << n;
        return;
    }
    cyfry(n / 10);
    cout << " " << n % 10;
}
```

CYFRY(4821) WYPISUJE

```
4 8 2 1
```

Zamieniamy kolejność dwóch linijek — i cyfry wychodzą od **pierwszej**!

↓ cyfry(4821) woła cyfry(482)
  ↓ cyfry(482) woła cyfry(48)
    ↓ cyfry(48) woła cyfry(4)
      ▸ cyfry(4): **4**
    ↑ cyfry(48) po powrocie: **8**
  ↑ cyfry(482): **2**
↑ cyfry(4821): **1**

09 Przykład: napis od tyłu

```
void odwroc(const string &s, int i) {
    if (i == s.size()) return;
    odwroc(s, i + 1);
    cout << s[i];
}

int main() {
    odwroc("kot", 0);
}
```

NA EKRANIE

```
tok
```

Tym razem parametr **i** to pozycja w napisie. Zaczynamy od 0, a każde wywołanie przesuwa się o jedną literę w prawo — aż do końca napisu.

- ↓ i = 0 (k) woła i = 1
  - ↓ i = 1 (o) woła i = 2
    - ↓ i = 2 (t) woła i = 3
      - ■ i = 3: koniec napisu
      - ▸ wypisuje **t**
    - ▸ wypisuje **o**
  - ▸ wypisuje **k**

09 Przykład: zapis dwójkowy

```
void dwojkowo(int n) {
    if (n < 2) {
        cout << n;
        return;
    }
    dwojkowo(n / 2);
    cout << n % 2;
}
```

DWOJKOWO(13) WYPISUJE

```
1101
```

Ostatnia cyfra zapisu dwójkowego to **n % 2**, a reszta to zapis liczby **n / 2**. Dokładnie jak cyfry od początku — tylko z 2 zamiast 10.

n	n / 2	n % 2
13	6	1
6	3	0
3	1	1
1	stop	wypisuje 1

Reszty wypisują się w drodze w górę, od dołu tabeli: **1, 1, 0, 1**.

10 Najczęstsze błędy (1/2)

01 **Brak warunku stopu.** Funkcja woła siebie bez końca, aż skończy się stos. Pisz warunek stopu jako **pierwszą linijkę** funkcji — zanim napiszesz cokolwiek innego.

02 **Krok w złą stronę.** `odliczaj(n + 1)` zamiast `odliczaj(n - 1)`: n rośnie i nigdy nie dojdzie do zera. Sprawdź, czy argument naprawdę **zbliża się** do warunku stopu.

03 **Warunek, który da się przeskoczyć.** Przy kroku `n - 2` warunek `n == 0` nie zadziała dla nieparzystych n. Bezpieczniej: `n <= 0`.

10 Najczęstsze błędy (2/2)

01 **Zapomniany return.** `if (n == 0) cout << "koniec";` — bez **return** funkcja idzie dalej i woła siebie dla -1, -2, … Warunek stopu musi **kończyć** funkcję.

02 **Zła kolejność.** Kod przed wywołaniem działa w drodze w dół, kod po wywołaniu — w drodze w górę. Jeśli liczby wychodzą odwrotnie, niż chcesz, przestaw linijki.

03 **Za głęboka rekurencja.** `odliczaj(10000000)` to 10 milionów pięter stosu — program się wysypie. Rekurencja głęboka na miliony to zadanie dla pętli.

11 Pytanie 1: co wypisze `f(4)`?

```
void f(int n) {
    if (n == 0) return;
    cout << n;
    f(n - 1);
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 1: 4321

- f(4): **4**
  - f(3): **3**
    - f(2): **2**
      - f(1): **1**
        - f(0): stop

NA EKRANIE

```
4321
```

Wypisywanie jest **przed** wywołaniem, więc liczby idą w drodze w dół: 4, 3, 2, 1. Bez spacji — bo w kodzie nie ma " ".

11 Pytanie 2: co wypisze `g(4)`?

```
void g(int n) {
    if (n == 0) return;
    g(n - 1);
    cout << n * n << " ";
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 2: 1 4 9 16

↓ g(4) woła g(3)
  ↓ g(3) woła g(2)
    ↓ g(2) woła g(1)
      ↓ g(1) woła g(0) — stop
      ▸ g(1): **1**
    ▸ g(2): **4**
  ▸ g(3): **9**
▸ g(4): **16**

NA EKRANIE

```
1 4 9 16
```

Wypisywanie jest **po** wywołaniu, więc kwadraty pojawiają się w drodze w górę — od najmniejszego n.

11 Pytanie 3: co wypisze `h(10)`?

```
void h(int n) {
    if (n <= 0) return;
    cout << n << " ";
    h(n - 3);
}
```

NA EKRANIE

?

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 3: 10 7 4 1

- h(10): **10**
  - h(7): **7**
    - h(4): **4**
      - h(1): **1**
        - h(−2): −2 ≤ 0, stop

NA EKRANIE

```
10 7 4 1
```

Po 1 przychodzi −2 — nie zero! Dobrze, że warunek to `n <= 0`, a nie `n == 0`.

11 Pytanie 4: znajdź błąd

```
void odliczaj(int n) {
    cout << n << " ";
    if (n == 0) return;
    odliczaj(n + 1);
}
```

NA EKRANIE

?

Program dla odliczaj(3) miał wypisać 3 2 1 0.

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 4: krok w złą stronę

BŁĄD

```
void odliczaj(int n) {
    cout << n << " ";
    if (n == 0) return;
    odliczaj(n + 1);
}
```

NA EKRANIE

```
3 4 5 6 7 8 9 10 …
Segmentation fault
```

POPRAWNIE

```
void odliczaj(int n) {
    cout << n << " ";
    if (n == 0) return;
    odliczaj(n - 1);
}
```

NA EKRANIE

```
3 2 1 0
```

Warunek stopu stoi tu **po** cout — dlatego wypisze się też 0. To nie błąd, tylko inna kolejność.

11 Pytanie 5: co wypisze `f(3)`?

```
void f(int n) {
    if (n == 0) return;
    f(n - 1);
    cout << n << " ";
    f(n - 1);
}
```

NA EKRANIE

?

Uwaga: tym razem funkcja woła siebie DWA razy.

Zastanów się i zapisz odpowiedź, **zanim** przejdziesz dalej.

11 Odpowiedź 5: 1 2 1 3 1 2 1

`f(n)` = najpierw całe `f(n - 1)`, potem **n**, potem znowu całe `f(n - 1)`.

Wywołanie	Wypisuje
`f(1)`	**1**
`f(2)`	f(1), 2, f(1) = **1 2 1**
`f(3)`	f(2), 3, f(2) = **1 2 1 3 1 2 1**

NA EKRANIE

```
1 2 1 3 1 2 1
```

Funkcja, która woła siebie **więcej niż raz**, to temat części 3 — zobaczysz tam całe **drzewo** wywołań.

12 Ściąga: rekurencja w jednym miejscu

Element	Co robi	Pamiętaj
`if (n == 0) return;`	warunek stopu — koniec schodzenia w dół	zawsze na górze funkcji
`f(n - 1);`	wywołanie dla mniejszego problemu	argument musi zbliżać się do stopu
kod **przed** f(…)	wykonuje się w drodze **w dół**	kolejność: 3, 2, 1
kod **po** f(…)	wykonuje się w drodze **w górę**	kolejność odwrotna: 1, 2, 3
stos wywołań	czekające kopie funkcji, każda z własnymi zmiennymi	za głęboko → przepełnienie
`n % 10, n / 10`	ostatnia cyfra i liczba bez niej	krok po cyfrach liczby
`f(s, i + 1)`	krok po pozycjach napisu / tablicy	stop, gdy i dojdzie do końca

12 Zadania na koniec — Twoja kolej

Wszystkie zadania są na **t-olimpians.com → Kompendium → Funkcje rekurencyjne → Krok 1**, z automatyczną sprawdzarką i wskazówką. Rób je po kolei:

1. Powtórz napis
2. Odliczanie do startu
3. Od 1 do n
4. Tam i z powrotem
5. Od a do b
6. Co druga liczba
7. Linia gwiazdek
8. Trójkąt z gwiazdek
9. Odwrócony trójkąt
10. Klepsydra
11. Schodki z liczb
12. Cyfry od końca
13. Cyfry od początku
14. Napis od tyłu
15. Tablica od końca
16. Zapis dwójkowy rekurencyjnie
17. Zapis w systemie o podstawie k

// KONIEC CZĘŚCI 1

Umiesz już napisać rekurencję!

Warunek stopu, mniejszy problem, kod przed i po wywołaniu, stos wywołań. Zrób zadania z kroku 1, a potem przejdź do części 2: funkcje rekurencyjne, które zwracają wynik.

STOS WYWOŁAŃ

`main()` program działa dalej

DNO STOSU